

zen-coding — Eval Results & Interview Brief

Domain-specific coding agent for quant research · eval harness & benchmark summary

Project in one sentence

zen-coding is a quant coding agent built as a thin, opinionated layer on the open-source agent harness pi (MIT). The harness is a pinned npm dependency; everything domain-specific lives in TypeScript extensions (guardrails, observability, modes, model registry, tools), a Python eval harness (scripts/run_eval.py + aggregate.py), and a Slack backend on the pi SDK.

Headline results

Benchmark	Baseline	With domain layer	Delta
paper-search skill (iter-1)	22.5% pass	100% pass	+77.5 pp
paper-extract skill (iter-1)	28.6% pass	100% pass	+71.4 pp
paper-replicate skill (iter-1)	64.3% pass	100% pass	+35.7 pp
zen-coding own suite (iter-3)	—	3/3 cases pass	all assertions green

Why this matters (FDE framing)

- Domain grounding is measurable: same tasks, same deterministic assertions — the skill layer lifts pass rate by 35–77 points while reducing token cost (paper-search: –26s, –2.2k tokens).
- Evals as product spec: pass criteria fixed before the run; assertions are deterministic bash on fresh checkouts at pinned commits — reproducible, no judge-model noise.
- The eval caught its own blind spot: iter-3 benchmark flagged that a new feature (OpenAlex expansion) was never exercised by any eval, and prescribed the fix — new prompts that don't name the source. Suite grew 2 -> 4 cases.
- Harness validation first: early iterations failed a trivial smoke test (agent exit 1 in ~3s) — a harness bug, not a model bug, caught because a smoke case existed.

Stated caveats

Skill A/B benchmarks used n=2 per config — directional, not precise. The zen-coding harness now supports --runs N (fresh checkout per attempt; per-case pass rate with binomial stddev, duration mean/stddev) and records the effective model/provider, pi version, and timestamp in run_meta.json. Skill A/B benchmarks live in the quant_research_lab repo's eval suite; zen-coding's own suite is evals/evals.json (3 enabled cases + 1 disabled placeholder).

zen-coding Eval Suite — Results by Iteration

scripts/run_eval.py · fresh checkout per case · deterministic bash assertions (exit 0 = pass)

Iteration	Cases run	Passed	Notes
iteration-1	smoke-hello	0/1	agent exit 1 after 3.8s — harness failure, no work done
iteration-2	3 cases	2/3	real cases pass; smoke-hello fails again (exit 1, 3.1s)
iteration-3	3 cases	3/3	harness fixed — full pass
iteration-4	smoke-hello (2 runs)	2/2 runs	multi-run verified; model provenance recorded (kimi-k3 / open

iteration-3 case detail (latest, all passing)

Case	Track	Duration	Assertions
smoke-hello	infra	8.3s	1/1
deploy-add-dry-run	deploy-strategy	32.8s	4/4
research-note-momentum-2022	quant-research	43.2s	4/4

What each case proves

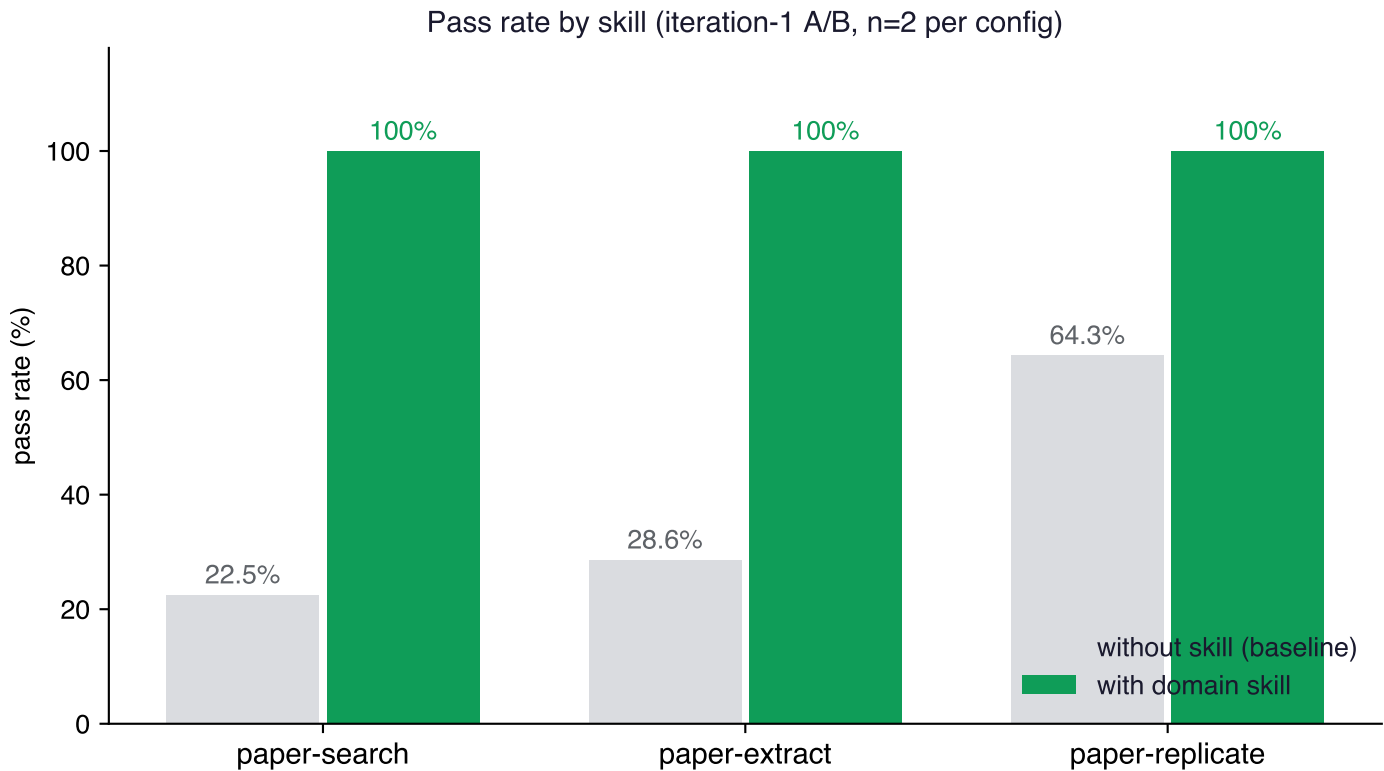
- smoke-hello — proves the loop works end-to-end (checkout -> headless run -> assertion). Exists precisely because iterations 1–2 showed you can't trust a benchmark until the harness itself is validated.
- deploy-add-dry-run — agent adds a --dry-run flag to a real deploy script. Assertions: still valid bash (bash -n), announces DRY RUN, never invokes git pull / docker compose up, existing behavior unchanged.
- research-note-momentum-2022 — agent writes a 200–400 word quant research note. Assertions: file exists, word count in range, contains a real citation, mentions a mitigation technique.

Harness design

Case = (repo, base_commit, prompt, assertions[]). Each attempt: clone repo at pinned commit -> run pi headless (npx pi -p) -> run assertions in the checkout -> result.json + full agent.jsonl transcript. --runs N repeats attempts on fresh checkouts (attempts/attempt-k/) and aggregates pass_rate +/- binomial stddev; a case passes only if ALL attempts pass. run_meta.json records effective model/provider (CLI flags, else pi settings.json defaults), pi version, runs_per_case, timestamp. Tracks group cases by workload (infra | deploy-strategy | quant-research) for per-track pass rates. aggregate.py folds an iteration into benchmark.json with per-case stats and regression diffing vs the previous iteration. Disabled placeholder case (newsletter-smoke) shows the enabled:false convention.

Skill A/B Benchmarks — Domain Layer vs. Baseline

quant_research_lab eval suite · with_skill vs. without_skill · deterministic assertions



Cost & latency detail (with_skill vs. baseline)

Skill	Δ pass rate	Δ time	Δ tokens
paper-search	+77.5 pp	−26.3s (faster)	−2.2k (cheaper)
paper-extract	+71.4 pp	+8.6s	−1.3k (cheaper)
paper-replicate	+35.7 pp	+34.3s	+0.8k

Reading: the domain skill is not a tax — on paper-search it is simultaneously much more reliable, faster, and cheaper. Where it costs more time (paper-replicate), the task is inherently longer (plan + backtest) and the baseline's speed came from failing early.

Interview line

“The base agent without domain context failed most of the time on these tasks; adding the skill layer took it to 100% while reducing token cost. That delta is the quantified value of forward-deployed domain work.”

Eval-Driven Iteration — the paper-search Skill over 5 Iterations

The benchmark as a feedback loop, including catching its own blind spot

iter-1 — Baseline A/B

with_skill 100% vs. without_skill 22.5%. Established that the skill matters at all.

iter-2 — Patch validation

SKILL.md patch validated: bare-id path ~26% faster, 5 fewer tool calls (root cause: http-to-https redirect bug). A 419s outlier on keyword-search was correctly attributed to arxiv 503/429 rate-limiting — environmental noise, not a skill regression. Verdict: safe to promote to team_skills.

iter-3 — The eval catches its own blind spot

New OpenAlex-primary skill: non-regressive on arxiv-coded prompts (both configs 100%). But the benchmark itself flagged: “the OpenAlex feature was not actually exercised in either eval — every prompt says 'arxiv' explicitly, so the scope expansion is invisible in this benchmark.” Prescribed fix: prompts that don't name a source.

iter-4 / iter-5 — Closing the gap

Suite grew 2 -> 4 cases (date-range filter, SSRN-heavy-topic honesty, topic-no-source-bias). Stable at 95.8% mean pass rate. Old-skill parity confirmed (no regression from the expansion).

Gap analysis (state before being asked)

Closed in the latest iteration:

- Model provenance — run_meta.json now records effective model/provider (CLI or settings.json defaults), pi version, runs_per_case, timestamp. Unblocks cross-model cost/latency benchmarking.
- Variance — --runs N executes each case N times on fresh checkouts; reports per-case pass_rate +/- binomial stddev and duration mean/stddev. Verified on iteration-4 (smoke-hello 2/2 runs, 8.2s +/- 0.4s).

Still open:

- No eval exercises the quant guardrail layer itself — the differentiating feature is still unmeasured (want: guardrail trigger rate, blocked-violation attempts, false positives).
- Tokens recorded but not \$ cost per passed task; failure taxonomy (harness vs model vs assertion) not encoded; small suite, no public-benchmark anchor; deterministic bash only — open-ended tasks need a rubric judge validated against human labels.

One-paragraph summary (memorize)

“I built deterministic, reproducible evals — fresh checkout, pinned commit, bash assertions, fixed before the run. Two results I'd highlight: A/B benchmarking showed the domain skill layer lifts pass rate from ~29% to 100% on paper-extraction tasks while cutting token cost — the quantified value of domain grounding. And the suite caught its own blind spot: it proved the skill didn't regress on arxiv prompts but couldn't validate the new OpenAlex expansion, so I added evals targeting exactly that gap. To me that's what good eval infrastructure looks like — it tells you what it can't measure.”